Pet Store Project: Building a Full-Stack Pet Management System

Section 1: Learn

What is the Pet Store Project?

The Pet Store Project is a real-world Java development project that teaches how to build a full-stack backend application using Spring Boot, JPA, and MySQL. It includes everything from entity creation to validation, repositories, and database integration.

Why Build This Project?

- **Hands-On Practice**: Reinforces Java and Spring Boot skills.
- **Real-World Structure**: Implements layered architecture, entity mapping, and DTO separation.
- **Professional Development**: Prepares learners for real-life Java backend roles.

How Does the Project Work?

- Uses **Spring Boot** for application logic.
- Uses **JPA** for entity persistence.
- Uses **MySQL** as the database.
- Uses **DTOs and Validation** to ensure clean architecture and input safety.

Interesting Anecdote

This type of project mimics the backend architecture used in real e-commerce and inventory systems. Concepts like user and pet entities, validation, and layered architecture are widely used in actual Java-based applications.

Section 2: Practice

Step-by-Step Project Guide

Step 1: Create Spring Boot Project

- Use Spring Initializr
- Add dependencies:
    - Spring Web
    - Spring Data JPA
    - MySQL Driver
    - Validation API

Step 2: Configure Database (application.yml)

```
spring:
 datasource:
  url: jdbc:mysql://localhost:3306/petstore_db
  username: your_username
  password: your_password
 jpa:
  hibernate:
   ddl-auto: update
  show-sql: true
  database-platform: org.hibernate.dialect.MySQL5Dialect
```

Step 3: Create Entity Classes

```
@Entity
public class Pet {
```

```
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    @Column(nullable = false)
    private String name;

    // Getters and setters
}
```

Step 4: Create User Entity and DTO

- **User.java**

```
@Entity
public class User {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private String name;
    private String email;
    private String password;
    // Getters and setters
}
```

- **UserDTO.java**

```
public class UserDTO {
    @NotNull(message = "Name required")
```

```
    private String name;

    @Email(message = "Enter a valid email")
    private String email;

    @Size(min = 6, message = "Password must be at least 6 characters")
    private String password;
}
```

Step 5: Create Repository Interfaces

```
@Repository
public interface PetRepository extends JpaRepository<Pet, Integer> {}

@Repository
public interface UserRepository extends JpaRepository<User, Integer> {
    User findByEmail(String email);
}
```

Step 6: Run Your Application

- Right-click on the main class containing @SpringBootApplication
- Choose **Run As > Java Application**
- Check logs for startup confirmation

Section 3: Know More

FAQs

Q1: Why use DTOs instead of directly using Entity classes?

- **Answer:** DTOs prevent overexposure of internal fields, isolate validation logic, and help maintain a clean architecture.

Q2: What does ddl-auto: update mean in Hibernate config?

- **Answer:** It tells Hibernate to automatically update the database schema based on entity classes.

Q3: Do I need to write SQL for saving data?

- **Answer:** No, Spring Data JPA provides methods like save() and findAll() by default.

Q4: How does validation work in this project?

- **Answer:** The @Valid annotation in controller methods triggers validation rules defined in DTO fields using annotations like @NotNull, @Size, etc.

Q5: Can I use Lombok in this project?

- **Answer:** Yes, you can use @Getter, @Setter, and @Data annotations from Lombok to reduce boilerplate code.